

TECHNICAL
PRESENTATION

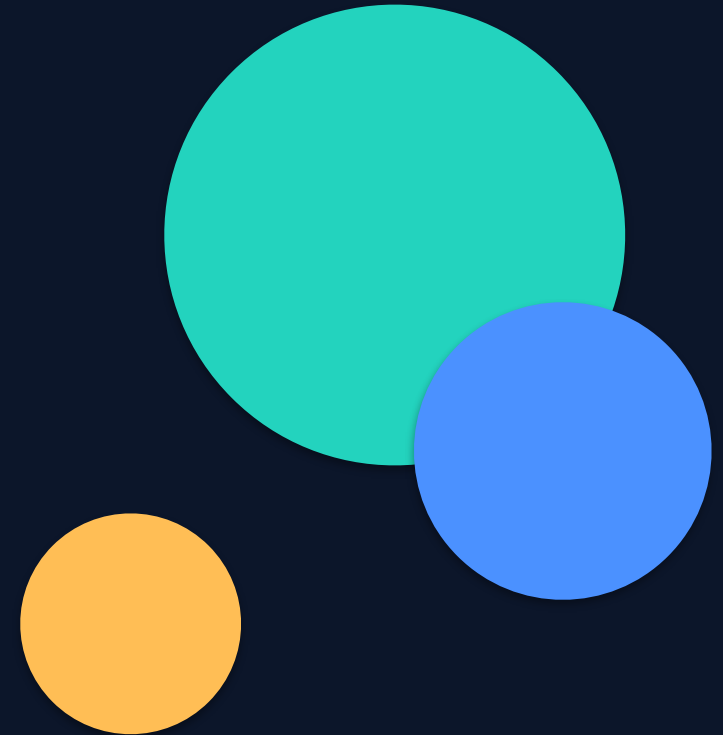
Linux Resource & Device Management

How the operating system coordinates shared resources, system calls, and kernel access in a multi-user server.

Presented by:

T.Subahan

192511121



The scenario: many users, shared hardware

A Linux server may handle these at once:

- Users running shells, scripts, and applications
- Services reading and writing shared files
- Processes competing for CPU time and memory
- Programs requesting disks, terminals, network devices, and other I/O

Core challenge

Share resources fairly while preserving isolation and system stability.

OS responsibility

Control access, coordinate operations, and enforce permissions.

Desired outcome

Reliable throughput, security, responsiveness, and predictable behavior.

OS functions involved in resource management

CPU scheduling

Chooses which runnable process or thread executes next; supports time-sharing and priorities.

Memory management

Maps virtual memory, isolates address spaces, reclaims pages, and handles allocation pressure.

File-system management

Organizes files and directories, checks permissions, and coordinates buffered storage I/O.

Device & I/O management

Uses drivers, queues, interrupts, and buffering to coordinate hardware access.

Protection & security

Applies users, groups, permissions, capabilities, and isolation boundaries.

Accounting & limits

Tracks resource use; cgroups and limits can constrain CPU, memory, and I/O.

How Linux shares resources safely



The kernel arbitrates access instead of allowing applications to control hardware directly.

Isolation means one user's process normally cannot read another user's memory or bypass file permissions. Shared resources are mediated through controlled kernel interfaces.

File system calls: from path to data

Common Linux/POSIX interfaces

open() / openat()	Open a file and receive a file descriptor.
read() / write()	Transfer bytes through an open descriptor.
close()	Release a descriptor when finished.
stat() / fstat()	Read metadata such as size, type, and permissions.
mkdir() / unlink()	Create a directory or remove a directory entry.
chmod() / chown()	Change mode bits or ownership, subject to privilege.

Important concept: file descriptor

An integer handle returned by `open()`. It refers to an open file description and is used by `read()`, `write()`, and `close()`.

Security check

The kernel evaluates credentials, directory traversal, permissions, and other access controls.

Device access and I/O system calls

read() / write()

Transfer data to or from a device exposed through a file descriptor.

ioctl()

Issue device-specific control requests, where supported by the driver.

poll() / select()

Wait until one or more descriptors are ready for I/O.

Typical I/O path



Device nodes often live under /dev. Access depends on permissions and device policy.

User process ↔ kernel interaction



Key distinction

A system call is a controlled entry into the kernel—not the same thing as an ordinary function call.

Example: a process reads a protected file

```
int fd = open("/var/log/app.log", O_RDONLY);  
if (fd >= 0) {  
    char buf[256];  
    ssize_t n = read(fd, buf, sizeof(buf));  
    close(fd);  
}
```

What the kernel does

- Resolves the path through the file system
- Checks credentials and read permission
- Returns a file descriptor or an error
- Copies available bytes to the process buffer
- Releases the descriptor on close()

Production code should also handle errors, partial reads, and interrupted system calls.